

Cooperative Coevolution for Resource-Constrained Agentic LLM Post-Training

Zhiyuan Wang¹, Shengcai Liu^{1*}, Jiahao Wu^{1,2}, Ning Lu^{1,3}, Hui Ouyang^{1,5},
Shaofeng Zhang^{1,4}, Haoze Lv^{1,4}, Ke Tang^{1*}

¹ Guangdong Provincial Key Laboratory of Brain-inspired Intelligent Computation, Department of Computer Science and Engineering, Southern University of Science and Technology

² Hong Kong Polytechnic University ³ Hong Kong University of Science and Technology

⁴ Zhongguancun Academy ⁵ Shenzhen Loop Area Institute

wangzy2020@mail.sustech.edu.cn, liusc3@sustech.edu.cn,
{12068041,12150066,12545007,12445025,11912814}@mail.sustech.edu.cn, tangk3@sustech.edu.cn

Abstract

Tool-using large language model (LLM) agents produce long, multi-turn trajectories, making gradient-based post-training memory-intensive. Evolution strategies (ES) enable memory-efficient full-parameter post-training without backpropagation and can eventually match the performance of gradient-based reinforcement learning (RL). However, resource-constrained settings typically offer only a few GPUs, so the high GPU-hour requirements of ES translate into prohibitively long training times. To address this, we introduce Cooperative Parameter-subspace Evolution Strategy (CoPES), a cooperative coevolutionary method that decomposes the full parameter space into lower-dimensional subspaces and searches over them cooperatively to improve optimization efficiency. We post-train a Qwen3.5-4B tool-using agent for the math task and evaluate it on five benchmarks of varying difficulty. Under the GPU-hour budget of full-parameter GRPO’s best validation checkpoint, CoPES recovers 92% of GRPO’s validation-accuracy gain, versus 67% for standard ES, while its theoretical GPU memory requirement is less than one-eighth that of full-parameter GRPO. It consistently outperforms standard ES and LoRA-based GRPO on all evaluated pass@*k* metrics across the five benchmarks. Additional experiments further show the advantage of CoPES on the question-answering task. These results demonstrate an improved trade-off between memory requirements and training time for agentic LLM post-training under resource constraints. The code is open-sourced in <https://github.com/MetaronWang/CoPES>

Introduction

Tool-using large language model (LLM) agents solve complex tasks by interleaving language generation with external actions and environmental feedback (Yao et al. 2023; Jiang et al. 2025; Lv et al. 2026). Post-training with verifiable rewards offers a scalable way to improve such behavior without requiring annotated trajectories (Lu et al. 2025). Reinforcement learning (RL) has consequently become a mainstream paradigm for agentic post-training, with current approaches dominated by gradient-based methods (Zhang et al. 2026; Wu et al. 2026). However, unlike models solving conventional single-turn tasks, tool-using agents often produce long, multi-turn trajectories. Backpropagating through these tra-

jectories creates a substantial GPU memory bottleneck for agentic LLM post-training.

Gradient-based post-training must store model weights, gradients, optimizer states, and the intermediate activations required for backpropagation. Prior work has proposed several approaches to reduce GPU memory usage during post-training. Group Relative Policy Optimization (GRPO) eliminates the need for a learned critic (Shao et al. 2024), while low-rank adaptation (LoRA) reduces gradient and optimizer-state memory by updating only low-rank adapters (Hu et al. 2022). However, neither avoids backpropagation through the policy model and the resulting need to store intermediate activations. Because activation memory grows with context length, this unresolved cost becomes especially severe for agentic post-training with long, multi-turn trajectories (Korthikanti et al. 2023). In practice, high-memory GPUs are costly, and obtaining them at scale is often difficult. These constraints motivate our focus on resource-constrained settings with limited per-GPU memory and only a small number of available GPUs, where the activation cost of long trajectories can make gradient-based agentic LLM post-training difficult to deploy.

Evolution strategies (ES) alleviate this memory bottleneck by avoiding backpropagation. Standard ES evaluates randomly perturbed models using scalar rewards and aggregates the results into a parameter update, requiring only forward generation and reward evaluation (Salimans et al. 2017). It can therefore optimize all model parameters without storing parameter gradients or backpropagation activations, substantially reducing GPU memory usage (Sun et al. 2026). Recent work shows that ES can eventually achieve performance comparable to gradient-based RL in LLM post-training (Qiu et al. 2026), but typically requires substantially more GPU-hours to do so (Sun et al. 2026). Moreover, existing studies focus primarily on single-turn reasoning tasks. In agentic settings, the high cost of generating each long, multi-turn trajectory further magnifies this computational disadvantage. Although ES evaluations are highly parallelizable, resource-constrained settings provide too few GPUs to absorb the additional GPU-hours through parallelism, resulting in impractically long wall-clock training times. The central challenge is therefore to improve the optimization efficiency of ES without sacrificing its memory advantage.

*Corresponding author.

To address this challenge, we propose a cooperative co-evolutionary method (Potter and De Jong 1994) for ES-based LLM post-training, termed Cooperative Parameter-subspace Evolution Strategy (CoPES). High dimensionality often reduces the effectiveness of evolutionary optimization, motivating cooperative coevolution to divide the search space into smaller subproblems (Yang, Tang, and Yao 2008). CoPES applies this principle to full-parameter ES by cooperatively searching lower-dimensional parameter subspaces. By confining each perturbation to one subspace, each scalar reward reflects a lower-dimensional change under a shared full-model context rather than simultaneous changes across the entire parameter space, allowing the fixed evaluation budget to be used more effectively. CoPES scales perturbations by subspace dimensionality to match the expected squared norm of a full-space perturbation, jointly standardizes rewards across subspaces, and synchronously composes the resulting updates. Together, these designs improve parameter exploration under fixed evaluation and GPU-hour budgets while retaining the forward-only, memory-efficient, and full-parameter nature of standard ES.

Our contributions are threefold:

- We introduce CoPES, which combines parameter-space decomposition and joint reward standardization for ES-based LLM post-training. To our knowledge, CoPES is the first cooperative coevolutionary method for full-parameter agentic LLM post-training.
- We show that CoPES improves fixed-budget optimization efficiency while preserving the memory-efficient, full-parameter advantages of ES, consistently outperforming standard ES and LoRA-based GRPO within the GPU-hour budget of full-parameter GRPO’s best validation checkpoint.
- Experiments on five math and three question-answering benchmarks, together with hardware-feasibility tests and controlled ablations, validate CoPES across tasks and multiple metrics while isolating the effects of its key components.

Related Work

Agentic LLM Post-Training

Recent agentic reinforcement learning methods optimize multi-turn tool-use trajectories using outcome-based or verifiable rewards (Zhang et al. 2026; Jiang et al. 2026). For example, Search-R1 trains LLMs to issue multiple search queries during reasoning and incorporate retrieved evidence (Jin et al. 2025), while ReTool combines a supervised cold start with outcome-driven RL to learn when and how to invoke a code interpreter (Feng et al. 2025). These methods commonly build on gradient-based policy optimization. GRPO removes the learned critic by estimating advantages from groups of sampled responses (Shao et al. 2024), and LoRA reduces the number of trainable parameters through low-rank adapters (Hu et al. 2022). Neither approach eliminates policy-model backpropagation. CoPES instead avoids policy-model backpropagation while optimizing all model parameters, addressing the broader GPU memory overhead

of gradient-based post-training that critic-free and parameter-efficient methods only partially reduce.

Evolution Strategies for LLM Post-Training

Evolution strategies estimate parameter updates from the rewards of randomly perturbed models and therefore do not backpropagate through sampled trajectories (Wierstra et al. 2014; Salimans et al. 2017). Recent work has demonstrated that evolutionary optimization can scale directly to billion-parameter LLMs (Qiu et al. 2026; Liu et al. 2025). ES-SAM combines ES with sharpness-aware maximization to improve generalization on mathematical reasoning while requiring only inference-level GPU memory (Sun et al. 2026). EGGROLL instead represents individual perturbations with low-rank factors to support batched evaluation at large population sizes (Sarkar et al. 2026). Collectively, these studies establish evolutionary optimization as a viable backpropagation-free alternative for LLM post-training. Nevertheless, standard ES is computationally intensive, often requiring substantial GPU-hours (Sun et al. 2026). In resource-constrained settings, limited parallelism translates this computational demand into long wall-clock training times. CoPES targets this efficiency bottleneck in agentic LLM post-training through cooperative parameter-subspace searches while retaining the forward-only, full-parameter nature of standard ES.

Cooperative Coevolution

Cooperative coevolution was introduced to evolve complex solutions as interacting, coadapted subcomponents (Potter and De Jong 1994, 2000). For continuous optimization, it decomposes a high-dimensional decision vector into subspaces, optimizes the corresponding subproblems, and evaluates each partial solution within a context supplied by the remaining subcomponents. Subsequent studies developed a large-scale optimization framework with random grouping and adaptive weighting (Yang, Tang, and Yao 2008) and introduced differential grouping to identify interacting variables and construct subproblems that better reflect variable interdependence (Omidvar et al. 2014). Recent work has extended this paradigm to 1.7-million-dimensional neural policy search and scalable many-objective optimization (Yang et al. 2022; Hu, Liu, and Yao 2024; Qian et al. 2025). CoPES builds on this literature by adapting cooperative coevolution to full-parameter ES-based LLM post-training.

Methodology

Building on cooperative coevolution (Potter and De Jong 1994), we propose Cooperative Parameter-subspace Evolution Strategy (CoPES) for full-parameter ES-based agentic LLM post-training. At each training step, CoPES randomly partitions the parameter space into equally sized subspaces, allocates a fixed total of N perturbations across them, and evaluates every subspace perturbation in the context of the current full model. It then jointly standardizes the resulting rewards and composes the subspace directions into a full-parameter update. Figure 1 illustrates this workflow. We

first formalize agentic post-training and the standard ES update, then present the three central designs of CoPES and its memory-efficient implementation.

Problem Formulation and Evolution Strategies

Agentic Post-Training Objective. Given a task prompt $x \sim \mathcal{D}$, an LLM policy π_θ interacts with a tool environment $\mathcal{E}$ to produce a multi-turn trajectory $\tau = (x, a_1, o_1, \dots, a_T, o_T)$, where actions are sampled from the policy and observations are returned by the environment. A verifier assigns the completed trajectory a scalar reward $R(\tau)$. The post-training objective is

$$J(\theta) = \mathbb{E}_{x \sim \mathcal{D}, \tau \sim (\pi_\theta, \mathcal{E})} [R(\tau)], \quad (1)$$

where $\theta \in \mathbb{R}^d$ contains all trainable model parameters. The post-training algorithm only observes completed trajectories and their scalar rewards; it does not require gradients through either the environment or the reward function. For an optimization algorithm $\mathcal{A}$ and a GPU-hour budget B , let

$$\theta_B = \mathcal{A}(\theta_0, \mathcal{D}, B) \quad (2)$$

denote the parameters obtained within that budget. Our objective is to maximize $J(\theta_B)$ under a fixed GPU-hour budget B . The experimental protocol defines how B is measured across methods, while the derivations below use the per-step perturbation budget to compare standard ES and CoPES.

Standard Evolution Strategy. Following the one-sided ES update adopted for full-parameter LLM post-training (Salimans et al. 2017; Qiu et al. 2026), standard ES samples N independent Gaussian directions for the current parameters θ and constructs

$$\epsilon_i \sim \mathcal{N}(\mathbf{0}, \mathbf{I}_d), \quad \theta_i = \theta + \sigma \epsilon_i, \quad (3)$$

where σ is the full-space perturbation scale. Each perturbed model generates one trajectory $\tau_{i,x}$ for every $x \in \mathcal{B}$, producing the batch-mean reward

$$r_i = \frac{1}{|\mathcal{B}|} \sum_{x \in \mathcal{B}} R(\tau_{i,x}). \quad (4)$$

ES standardizes these N rewards using

$$\begin{aligned} \mu_r &= \frac{1}{N} \sum_{i=1}^N r_i, & s_r &= \sqrt{\frac{1}{N} \sum_{i=1}^N (r_i - \mu_r)^2}, \\ \hat{r}_i &= \frac{r_i - \mu_r}{s_r}, \end{aligned} \quad (5)$$

and applies

$$\theta^+ = \theta + \frac{\alpha}{N} \sum_{i=1}^N \hat{r}_i \epsilon_i, \quad (6)$$

where α is the effective update step size and is set independently of σ . This update uses no mirrored samples, unperturbed-model baseline, momentum, or gradient-based optimizer. With a limited population, each sampled direction spans the entire d -dimensional parameter space and is evaluated through a single scalar reward. This motivates decomposing the search while keeping the total number of perturbation evaluations fixed.

Cooperative Parameter-Subspace Evolution Strategy

Cooperative Subspace Search. CoPES replaces the monolithic d -dimensional search with K cooperative searches over lower-dimensional subspaces. At every training step, it samples a uniform random partition $\{\mathcal{S}_k\}_{k=1}^K$ of all parameter indices such that

$$\begin{aligned} \mathcal{S}_k \cap \mathcal{S}_\ell &= \emptyset \ (k \neq \ell), & \bigcup_{k=1}^K \mathcal{S}_k &= \{1, \dots, d\}, \\ d_k &= |\mathcal{S}_k| = \frac{d}{K}. \end{aligned} \quad (7)$$

We assume that d and N are divisible by K to simplify notation. The partition is resampled each step so that parameters are not permanently restricted to a fixed grouping. Let $P_k : \mathbb{R}^{d_k} \rightarrow \mathbb{R}^d$ embed a subspace vector into the full parameter space. CoPES allocates $N_k = N/K$ perturbations to each subspace and constructs

$$\epsilon_{k,i} \sim \mathcal{N}(\mathbf{0}, \mathbf{I}_{d_k}), \quad \theta_{k,i} = \theta + \sigma_k P_k \epsilon_{k,i}. \quad (8)$$

Only parameters in $\mathcal{S}_k$ are perturbed; all remaining parameters retain their current values and provide a shared full-model context. Each perturbed model is evaluated on the same batch $\mathcal{B}$, yielding

$$r_{k,i} = \frac{1}{|\mathcal{B}|} \sum_{x \in \mathcal{B}} R(\tau_{k,i,x}). \quad (9)$$

Although each subspace receives only N/K perturbations, its search dimension is reduced from d to d/K . More importantly, each scalar reward is associated with a perturbation confined to one subspace under a shared full-model context, rather than with simultaneous perturbations across the entire parameter space. Because the subspaces have disjoint supports, CoPES obtains K mutually orthogonal subspace estimates and composes them into a full-parameter update.

Dimension-Aware Perturbation Scale. Using the full-space scale unchanged would reduce the magnitude of a subspace perturbation simply because it contains fewer dimensions. For $\delta = \sigma \epsilon$ and $\delta_k = \sigma_k P_k \epsilon_k$, their expected squared norms are

$$\mathbb{E}[\|\delta\|_2^2] = d\sigma^2, \quad \mathbb{E}[\|\delta_k\|_2^2] = d_k \sigma_k^2. \quad (10)$$

Matching these quantities gives

$$\sigma_k = \sigma \sqrt{\frac{d}{d_k}} = \sqrt{K} \sigma. \quad (11)$$

This adjustment matches the expected squared perturbation norm of full-space ES while retaining the same update step size α .

Joint Reward Standardization and Cooperative Update. After evaluating all $N = \sum_k N_k$ perturbed models, CoPES pools their batch-mean rewards and computes

$$\begin{aligned} \mu_c &= \frac{1}{N} \sum_{k=1}^K \sum_{i=1}^{N_k} r_{k,i}, & s_c &= \sqrt{\frac{1}{N} \sum_{k=1}^K \sum_{i=1}^{N_k} (r_{k,i} - \mu_c)^2}, \\ \hat{r}_{k,i} &= \frac{r_{k,i} - \mu_c}{s_c}. \end{aligned} \quad (12)$$

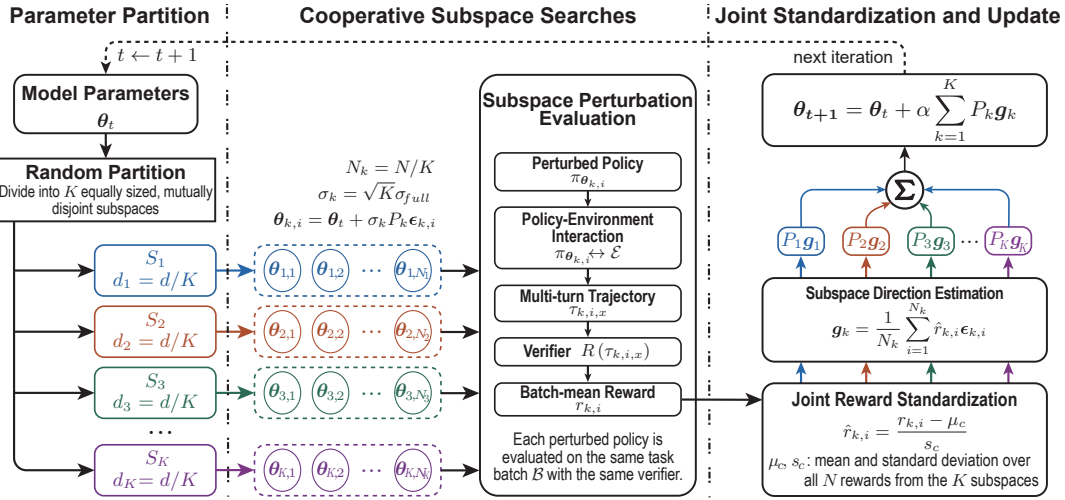


Figure 1: Overview of CoPES. At each training step, the model parameters are randomly partitioned into K disjoint subspaces, and N/K perturbations are evaluated within each subspace under a shared full-model context. Rewards from all subspaces are jointly standardized, and the resulting subspace estimates are composed into a full-parameter update.

Independent standardization would estimate a separate mean and standard deviation from only N_k samples in each subspace. Joint standardization instead uses all N rewards to estimate a shared mean and standard deviation, providing more stable normalization statistics and allowing the subspace directions to be combined on a common reward scale. The shared training batch, matched expected squared perturbation norm, and common reward function provide a consistent evaluation basis across subspaces.

CoPES estimates each subspace direction and composes the full update as

$$g_k = \frac{1}{N_k} \sum_{i=1}^{N_k} \hat{r}_{k,i} \epsilon_{k,i}, \quad \theta^+ = \theta + \alpha \sum_{k=1}^K P_k g_k. \quad (13)$$

All perturbed models are evaluated and their rewards are standardized before any parameter update is applied. Consequently, every g_k is estimated from the same pre-update model, and their composition is a synchronous full-parameter update at the algorithmic level. Algorithm 1 summarizes one CoPES training step.

Memory-Efficient Implementation and Cost

Seed Replay, Chunked Processing, and Weight Backup.

Following OpenAI-ES (Salimans et al. 2017), CoPES represents each perturbation by a random seed and regenerates its Gaussian direction during the update, avoiding storage of full perturbation vectors; model parameters are processed in chunks. Unlike prior work that restores parameters by subtracting replayed perturbations, CoPES backs up the pre-perturbation weights in comparatively inexpensive CPU memory and restores them after each perturbed-model evaluation, avoiding residual errors from the non-reversibility of floating-point addition and subtraction and ensuring that every perturbation is evaluated from the same pre-update model. CoPES also stores a partition seed and replays the

Algorithm 1: CoPES

Input: Parameters θ , shared batch $\mathcal{B}$, reward R , population N , and subspace count K

Parameters: Full-space scale σ and step size α

Output: Updated parameters θ^+

- 1: Sample an equal-size random partition $\{\mathcal{S}_k\}_{k=1}^K$
- 2: Set $d_k = d/K$, $N_k = N/K$, and $\sigma_k = \sqrt{K} \sigma$
- 3: **for** $k = 1, \dots, K$ **do**
- 4: **for** $i = 1, \dots, N_k$ **do**
- 5: Sample $\epsilon_{k,i} \sim \mathcal{N}(\mathbf{0}, \mathbf{I}_{d_k})$
- 6: Set $\theta_{k,i} = \theta + \sigma_k P_k \epsilon_{k,i}$
- 7: Generate $\tau_{k,i,x}$ for every $x \in \mathcal{B}$
- 8: Compute $r_{k,i}$ using Equation (9)
- 9: **end for**
- 10: **end for**
- 11: Jointly standardize all $r_{k,i}$ using Equation (12)
- 12: **for** $k = 1, \dots, K$ **do**
- 13: Compute g_k using Equation (13)
- 14: **end for**
- 15: $\theta^+ \leftarrow \theta + \alpha \sum_{k=1}^K P_k g_k$
- 16: **return** θ^+

chunk-to-subspace assignments, avoiding a parameter-level mask. These mechanisms support stable, memory-efficient execution rather than constituting separate algorithmic contributions.

Evaluation and Memory Costs. CoPES matches standard ES in using N perturbed-model evaluations and the same number of generated trajectories per step; each perturbed model generates a trajectory for every prompt in $\mathcal{B}$. Thus, decomposition does not reduce the cost of an individual evaluation but reallocates the fixed perturbation budget across lower-dimensional subspaces. Like standard ES, CoPES requires only forward generation and reward evaluation, storing

no parameter gradients, optimizer states, or backpropagation activations.

Experiments

We organize the main experiments around three research questions. **RQ1:** In agentic LLM post-training, can ES-based methods match gradient-based RL in performance, and how do their memory efficiency and GPU-hour cost compare? **RQ2:** Under resource constraints, how does CoPES compare with standard ES and GRPO variants in hardware feasibility and fixed-budget performance? **RQ3:** How do the number of subspaces and joint reward standardization affect CoPES? We study these questions on the math task evaluated across five benchmarks and conclude with additional experiments on multi-hop question-answering.

Experimental Setup

Models and Tasks. We post-train Qwen3.5-4B (Qwen Team 2026) on the math task and multi-hop question-answering (QA) task. Our main experiments focus on math, while the QA experiments examine the applicability of CoPES to a different task. For math, we post-train on the MATH training set (Hendrycks et al. 2021) and evaluate on AIME 2024¹, AIME 2025², GSM8K (Cobbe et al. 2021), MATH-500 (Lightman et al. 2024), and MATH-Test; MATH-500 is a curated subset of MATH-Test. For QA, we post-train on the HotpotQA training set and evaluate on HotpotQA, 2Wiki, and MuSiQue (Yang et al. 2018; Ho et al. 2020; Trivedi et al. 2022). Dataset descriptions and split details are provided in Supplementary A.

Baselines and Hyperparameters. We compare CoPES with Qwen3.5-4B and Qwen3.5-9B without post-training, full-parameter GRPO (Shao et al. 2024), LoRA-based GRPO (Hu et al. 2022), and standard ES. Following recommended settings from prior work, all post-training methods use a prompt batch size of 64. Full-parameter and LoRA-based GRPO use learning rates of 1×10^{-6} and 5×10^{-6} , respectively (Shao et al. 2024; Devulapalli, Satsangi, and Narayan 2025). Standard ES and CoPES both use $N = 40$ perturbations per step, $\sigma = 1 \times 10^{-3}$, and $\alpha = 5 \times 10^{-4}$ (Qiu et al. 2026; Liu et al. 2025). CoPES additionally uses $K = 4$, yielding $\sigma_k = \sqrt{K}\sigma = 2 \times 10^{-3}$. We evaluate model performance on a held-out validation set every 8 steps and select each method’s best observed validation checkpoint. Full-parameter GRPO’s selected step defines the fixed budget in RQ2. Other hyperparameters and implementation details are provided in Supplementary A.

Rewards and Metrics. The base task score is answer accuracy for math and F1 for QA. For both tasks, the training reward additionally includes a tool-use bonus for a positive task score and format penalties. We report pass@ k (Chen et al. 2021) for math based on repeated sampling. For QA, we report accuracy, exact match (EM), and F1, each averaged over 32 samples per problem. Since our experiments were

conducted on servers equipped with different GPU types, we report class-level GPU-hour estimates. We profile CoPES and full-parameter GRPO for 32 steps on the same machine as representatives of the ES-based and RL-based method classes, respectively, and extrapolate their average per-step costs to the reported training steps. Complete reward definitions, metric calculations, sampling protocols, and GPU-hour accounting are provided in Supplementary A.

Agent and Compute Environment. We use a tool-using agent equipped with Local Wiki Search over Wiki18 (Jin et al. 2025) and a Python Sandbox. GPU-hour profiling is performed on a server with $8 \times$ RTX 5880 Ada GPUs. To accelerate training and evaluation, we additionally use servers equipped with NVIDIA A30, RTX A6000, and A100 GPUs.

Memory and Computational Efficiency of ES

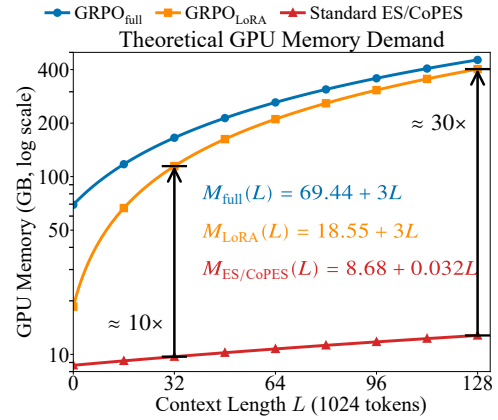


Figure 2: Theoretical GPU memory requirements versus context length with FlashAttention (Dao et al. 2022) and KV caching, excluding other memory optimizations.

Effectiveness in Agentic Post-Training. Table 1 shows that standard ES performs comparably to the RL-based methods at its selected checkpoint, demonstrating its viability for agentic post-training. Standard ES obtains the highest pass@1 on three of the five math benchmarks.

Theoretical Memory Efficiency. Figure 2 compares the theoretical GPU memory requirements of different methods on Qwen3.5-4B across context lengths. At 128K, the theoretical requirement of even LoRA-based GRPO is more than 30 $\times$ that of ES. These are accounting estimates rather than measured peak GPU memory; calculation details are provided in Supplementary B.

Computational Efficiency. Despite competitive performance and memory efficiency, standard ES reaches its selected checkpoint at substantially higher computational cost. As shown in Table 1, standard ES is estimated to require 480.60 GPU-hours, more than six times the estimated 78.61 GPU-hours for full-parameter GRPO. In resource-constrained settings with only one or two GPUs, this computational demand translates into impractically long wall-clock training times.

¹https://huggingface.co/datasets/HuggingFaceH4/aime_2024

²https://huggingface.co/datasets/test-time-compute/aime_2025

	Pass@1 (%) on Benchmarks					Train Steps	Est. GPU-hours	Valid Acc.	Run on a 24GB GPU
	AIME 2024	AIME 2025	GSM8K	MATH-500	MATH-Test				
<i>Reference Model</i>									
Qwen3.5-4B	17.13	35.04	9.22	57.44	59.02	-	-	49.20	-
<i>Validation-Selected Checkpoints</i>									
+ Full-param GRPO	44.28	58.68	94.19	89.71	91.66	16	78.61	91.47	×
+ LoRA-based GRPO	41.86	54.75	94.38	89.34	91.40	64	314.45	91.07	×
+ Standard ES	45.26	61.69	94.02	89.80	91.47	112	480.60	91.73	✓
<i>Post-Training for 16 Update Steps (Comparable Estimated GPU-hours)</i>									
+ LoRA-based GRPO	28.14	40.76	47.49	77.19	79.18	16	78.61	73.60	×
+ Standard ES	31.33	44.45	46.88	78.88	80.43	16	68.65	77.60	✓
+ CoPES (ours)	38.39	52.75	92.57	88.29	90.16	16	68.65	88.00	✓
<i>Larger-model Reference (No Additional Post-Training)</i>									
Qwen3.5-9B	16.67	30.42	51.23	77.72	81.24	-	-	-	-

Table 1: Pass@1 (%) and training costs on five math benchmarks. The 16-step block matches update count and unique prompt batches; estimated GPU-hours are 78.61 for the GRPO variants and 68.65 for the ES variants. It is therefore not an exact GPU-hour-matched comparison. MATH-500 is a subset of MATH-Test.

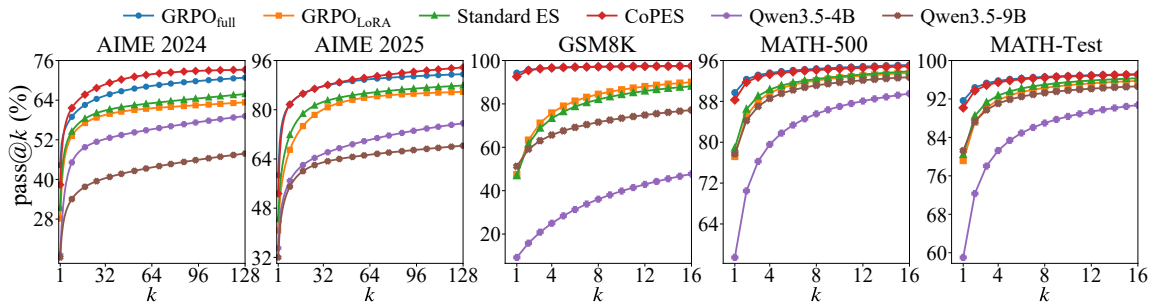


Figure 3: Pass@k of four post-training methods under the 16-step full-parameter GRPO budget, with Qwen3.5-4B and Qwen3.5-9B without post-training as references.

Method	1× 24GB	1× 48GB	4× 48GB	8× 48GB
Full-param GRPO	×	×	×	✓
LoRA-based GRPO	×	×	×	✓
Standard ES / CoPES	✓	✓	✓	✓

Table 2: Training feasibility with a 128K context limit. ✓ denotes successful training, whereas × indicates an out-of-memory failure.

Answering RQ1, standard ES can match RL-based performance with substantially less GPU memory but at considerably greater GPU-hour cost.

Feasibility and Performance under Resource Constraints

Feasibility across Hardware Configurations. We evaluate training feasibility on actual hardware with all memory optimizations supported by the respective frameworks enabled; implementation details are provided in Supplemen-

tary A. As shown in Table 2, the ES-based methods can be trained on a single 24 GB GPU, whereas, among the evaluated configurations under the 128K context setting, both GRPO variants succeed only on 8×48 GB GPUs. The smallest tested configuration supporting ES therefore provides less than one-eighth of the aggregate GPU memory capacity of the smallest tested configuration on which GRPO succeeds. Thus, when only one or a few GPUs are available, the GRPO-based methods are infeasible while the ES-based methods remain trainable.

Matched-step performance. In resource-constrained settings, a limited number of GPUs also limits the GPU-hours that can be spent within an acceptable wall-clock time. Full-parameter GRPO reaches its best observed validation checkpoint at step 16, as shown in Table 1. We therefore train all four post-training methods for 16 steps to ensure that they process the same number of training prompts while keeping their GPU-hour costs comparable. The ES-based methods have slightly lower estimated GPU-hour costs than the RL-based methods, but the difference is small. Under this budget, Table 1 and Figure 3 show that CoPES consistently outperforms standard ES and LoRA-based GRPO

Variant	AIME 2024	AIME 2025	GSM 8K	MATH 500	MATH Test
$K = 1$ (Std. ES)	31.33	44.45	46.88	78.88	80.43
$K = 2$	34.48	46.98	82.07	85.96	87.70
$K = 4$ (Default)	38.39	52.75	92.57	88.29	90.16
$K = 8$	16.61	26.56	37.32	58.07	59.38
Indep. Z-score	21.71	30.30	90.98	87.68	89.45
ES- 2σ	24.56	41.28	15.12	65.28	52.25

Table 3: Ablation results after 16 training steps (pass@1, %). K denotes the number of subspaces; Indep. Z-score uses per-subspace reward standardization; ES- 2σ is standard ES with twice the perturbation scale.

across the five benchmarks while remaining competitive with full-parameter GRPO. Relative to Qwen3.5-4B without post-training, CoPES recovers 92% of the validation-accuracy gain achieved by full-parameter GRPO, compared with 67% for standard ES.

Larger- k Results on AIME. Figure 3 shows that, as k increases, CoPES surpasses full-parameter GRPO on the challenging AIME 2024 and AIME 2025 benchmarks despite using the same fixed 16-step training budget.

Answering RQ2, CoPES retains the single-GPU feasibility of ES while outperforming standard ES and LoRA-based GRPO under the fixed budget; at larger k , it also surpasses full-parameter GRPO on AIME.

Ablation Studies

The Number of Subspaces. Holding all other settings fixed, we compare $K \in \{1, 2, 4, 8\}$ using pass@1 on the five benchmarks after 16 training steps. As K changes, we set $N_k = N/K$ and adjust the perturbation scale as $\sigma_k = \sqrt{K}\sigma$. Table 3 shows that performance improves as K increases from 1 to 4, but drops sharply at $K = 8$. At $K = 8$, the population per subspace falls to $N_k = 5$, while the parameter partition becomes highly fragmented; both factors may reduce update quality and degrade performance. For this setup, we recommend $N_k \geq 10$; this threshold may not generalize to other configurations.

Joint Reward Standardization. The Indep. Z-score row in Table 3 reports the results when perturbation rewards are standardized independently within each subspace. Joint reward standardization achieves higher pass@1 on every benchmark, validating its effectiveness.

Perturbation-Scale Control. To test whether the improvement of CoPES over standard ES is attributable solely to its larger per-coordinate perturbation scale ($\sigma_k = 2\sigma$), we additionally evaluate standard ES with the same scale under the fixed budget. The ES- 2σ row in Table 3 shows that increasing the perturbation scale of standard ES degrades its performance across the benchmarks rather than improving it.

Answering RQ3, a moderate subspace count and joint reward standardization improve CoPES, while scale control rules out larger perturbations as the sole source of its gains.

Benchmark	Metric	Qwen	GRPO	Std. ES	CoPES
2Wiki	Acc.	23.41	60.95	55.59	<u>59.06</u>
	EM	6.06	52.16	45.28	<u>48.29</u>
	F1	8.68	58.49	51.53	<u>55.50</u>
HotpotQA	Acc.	29.51	53.87	53.22	<u>53.30</u>
	EM	8.27	47.33	46.52	<u>46.96</u>
	F1	12.70	59.38	58.22	<u>58.83</u>
MuSiQue	Acc.	10.69	29.89	29.07	<u>29.68</u>
	EM	2.67	<u>23.07</u>	21.55	23.34
	F1	4.75	<u>33.28</u>	31.70	33.73

Table 4: Multi-hop QA performance (%). Qwen: Qwen3.5-4B without post-training; GRPO: full-parameter GRPO; Std. ES: standard ES; CoPES: ours. Post-training methods run for 48 steps. Bold and underlining indicate the best and second-best results, respectively.

Additional Experiments on Multi-hop QA

We further examine whether the effectiveness of CoPES extends beyond the math task. Full-parameter GRPO’s best observed validation checkpoint is at step 48, defining the common 48-step post-training budget. All other method-specific hyperparameters and agent settings remain the same as in the math task. As shown in Table 4, all three post-training methods substantially improve over Qwen3.5-4B across the multi-hop QA benchmarks.

CoPES outperforms standard ES on all nine benchmark-metric combinations, extending its advantage to another agentic task and reward signal. Against full-parameter GRPO, it achieves higher EM and F1 with comparable accuracy on MuSiQue, remains close across metrics on HotpotQA, and trails on 2Wiki. Together with its hardware feasibility, these results support CoPES as a practical option for resource-constrained QA post-training. We additionally evaluate accuracy-based pass@ k on QA; CoPES notably surpasses full-parameter GRPO on 2Wiki and MuSiQue for $k \geq 2$, as detailed in Supplementary E.

Conclusion

We introduced Cooperative Parameter-subspace Evolution Strategy (CoPES), a cooperative coevolutionary method that improves the fixed-budget optimization efficiency of ES-based agentic LLM post-training through parameter-subspace search and joint reward standardization. Experiments on math and multi-hop QA show that CoPES substantially improves over standard ES while approaching full-parameter GRPO, without sacrificing the memory-efficient, full-parameter nature of ES. These findings make ES-based agentic LLM post-training more practical under limited GPU resources. Current experiments focus on Qwen3.5-4B and two agentic tasks; future work will extend CoPES to a broader range of models and agent environments, with particular emphasis on adaptive parameter-space partitioning.

References

- Chen, M.; Tworek, J.; Jun, H.; Yuan, Q.; de Oliveira Pinto, H. P.; Kaplan, J.; Edwards, H.; Burda, Y.; Joseph, N.; Brockman, G.; Ray, A.; Puri, R.; Krueger, G.; Petrov, M.; Khlaaf, H.; Sastry, G.; Mishkin, P.; Chan, B.; Gray, S.; Ryder, N.; Pavlov, M.; Power, A.; Kaiser, L.; Bavarian, M.; Winter, C.; Tillet, P.; Such, F. P.; Cummings, D.; Plappert, M.; Chantzis, F.; Barnes, E.; Herbert-Voss, A.; Guss, W. H.; Nichol, A.; Paino, A.; Tezak, N.; Tang, J.; Babuschkin, I.; Balaji, S.; Jain, S.; Saunders, W.; Hesse, C.; Carr, A. N.; Leike, J.; Achiam, J.; Misra, V.; Morikawa, E.; Radford, A.; Knight, M.; Brundage, M.; Murati, M.; Mayer, K.; Welinder, P.; McGrew, B.; Amodei, D.; McCandlish, S.; Sutskever, I.; and Zaremba, W. 2021. Evaluating Large Language Models Trained on Code. *CoRR*, abs/2107.03374.
- Cobbe, K.; Kosaraju, V.; Bavarian, M.; Chen, M.; Jun, H.; Kaiser, L.; Plappert, M.; Tworek, J.; Hilton, J.; Nakano, R.; Hesse, C.; and Schulman, J. 2021. Training Verifiers to Solve Math Word Problems. *CoRR*, abs/2110.14168.
- Dao, T.; Fu, D. Y.; Ermon, S.; Rudra, A.; and Ré, C. 2022. FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness. In *Proceedings of NeurIPS 2022*, volume 35, 16344–16359.
- Devulapalli, U.; Satsangi, A.; and Narayan, A. 2025. Fine-Tuning Large Language Models for Structured Clinical Report Generation Using GRPO. *Proceedings of the AAAI Symposium Series*, 7(1): 496–500.
- Feng, J.; Huang, S.; Qu, X.; Zhang, G.; Qin, Y.; Zhong, B.; Jiang, C.; Chi, J.; and Zhong, W. 2025. ReTool: Reinforcement Learning for Strategic Tool Use in LLMs. *CoRR*, abs/2504.11536.
- Hendrycks, D.; Burns, C.; Kadavath, S.; Arora, A.; Basart, S.; Tang, E.; Song, D.; and Steinhardt, J. 2021. Measuring Mathematical Problem Solving With the MATH Dataset. In *Proceedings of NeurIPS 2021*, volume 1.
- Ho, X.; Nguyen, A.-K. D.; Sugawara, S.; and Aizawa, A. 2020. Constructing a Multi-hop QA Dataset for Comprehensive Evaluation of Reasoning Steps. In *Proceedings of COLING 2020*, 6609–6625.
- Hu, C.; Liu, J.; and Yao, X. 2024. Evolutionary Reinforcement Learning via Cooperative Coevolution. In *Proceedings of ECAI 2024*, volume 392, 3300–3307. IOS Press.
- Hu, E. J.; Shen, Y.; Wallis, P.; Allen-Zhu, Z.; Li, Y.; Wang, S.; Wang, L.; and Chen, W. 2022. LoRA: Low-Rank Adaptation of Large Language Models. In *Proceedings of ICLR 2022*.
- Jiang, C.; Shu, X.; Qian, H.; Lu, X.; Zhou, J.; Zhou, A.; and Yu, Y. 2025. LLMOPT: Learning to Define and Solve General Optimization Problems from Scratch. In *Proceedings of ICLR*.
- Jiang, D.; Lu, Y.; Li, Z.; Lyu, Z.; Nie, P.; Wang, H.; Su, A.; Chen, H.; Zou, K.; Du, C.; Pang, T.; and Chen, W. 2026. VerlTool: Towards Holistic Agentic Reinforcement Learning with Tool Use. *Trans. Mach. Learn. Res.*
- Jin, B.; Zeng, H.; Yue, Z.; Wang, D.; Zamani, H.; and Han, J. 2025. Search-R1: Training LLMs to Reason and Leverage Search Engines with Reinforcement Learning. In *Proceedings of COLM 2025*.
- Korthikanti, V. A.; Casper, J.; Lym, S.; McAfee, L.; Andersch, M.; Shoeybi, M.; and Catanzaro, B. 2023. Reducing Activation Recomputation in Large Transformer Models. In *Proceedings of MLSys 2023*.
- Kwon, W.; Li, Z.; Zhuang, S.; Sheng, Y.; Zheng, L.; Yu, C. H.; Gonzalez, J. E.; Zhang, H.; and Stoica, I. 2023. Efficient Memory Management for Large Language Model Serving with PagedAttention. In *Proceedings of SOSP 2023*, 611–626. ACM.
- Lightman, H.; Kosaraju, V.; Burda, Y.; Edwards, H.; Baker, B.; Lee, T.; Leike, J.; Schulman, J.; Sutskever, I.; and Cobbe, K. 2024. Let’s Verify Step by Step. In *Proceedings of ICLR 2024*.
- Liu, W.; Song, S.; Hao, H.; and Zhou, A. 2025. EA4LLM: A Gradient-Free Approach to Large Language Model Optimization via Evolutionary Algorithms. *CoRR*, abs/2510.10603.
- Lu, N.; Liu, S.; Wu, J.; Chen, W.; Zhang, Z.; Ong, Y.-S.; Wang, Q.; and Tang, K. 2025. Safe Delta: Consistently Preserving Safety when Fine-Tuning LLMs on Diverse Datasets. In *Proceedings of ICML 2025*.
- Lv, H.; Lu, N.; Zhou, Z.; and Liu, S. 2026. AHD Agent: Agentic Reinforcement Learning for Automatic Heuristic Design. *CoRR*, abs/2605.08756.
- Meurer, A.; Smith, C. P.; Paprocki, M.; Čertík, O.; Kirpichev, S. B.; Rocklin, M.; Kumar, A.; Ivanov, S.; Moore, J. K.; Singh, S.; Rathnayake, T.; Vig, S.; Granger, B. E.; Muller, R. P.; Bonazzi, F.; Gupta, H.; Vats, S.; Johansson, F.; Pedregosa, F.; Curry, M. J.; Terrel, A. R.; Roučka, Š.; Saboo, A.; Fernando, I.; Kulal, S.; Cimrman, R.; and Scopatz, A. 2017. SymPy: Symbolic Computing in Python. *PeerJ Comput. Sci.*, 3: e103.
- Omidvar, M. N.; Li, X.; Mei, Y.; and Yao, X. 2014. Cooperative Co-Evolution With Differential Grouping for Large Scale Optimization. *IEEE Trans. Evol. Comput.*, 18(3): 378–393.
- Potter, M. A.; and De Jong, K. A. 1994. A Cooperative Coevolutionary Approach to Function Optimization. In *Proceedings of PPSN 1994*, 249–257.
- Potter, M. A.; and De Jong, K. A. 2000. Cooperative Coevolution: An Architecture for Evolving Coadapted Subcomponents. *Evol. Comput.*, 8(1): 1–29.
- Qian, H.; Wu, Y.; Qin, R.; An, X.; Chen, Y.; and Zhou, A. 2025. Provable Space Discretization Based Evolutionary Search for Scalable Multi-Objective Security Games. *Swarm Evol. Comput.*, 92: 101770.
- Qiu, X.; Gan, Y.; Hayes, C. F.; Liang, Q.; Xu, Y.; Dailey, R.; Meyerson, E.; Hodjat, B.; and Miikkulainen, R. 2026. Evolution Strategies at Scale: LLM Fine-Tuning Beyond Reinforcement Learning. In *Proceedings of ICML 2026*, volume 306.
- Qwen Team. 2026. Qwen3.5: Towards Native Multimodal Agents.
- Salimans, T.; Ho, J.; Chen, X.; Sidor, S.; and Sutskever, I. 2017. Evolution Strategies as a Scalable Alternative to Reinforcement Learning. *CoRR*, abs/1703.03864.

Sarkar, B.; Fellows, M.; Duque, J. A.; Letcher, A.; Villares, A. L.; Sims, A.; Wibault, C.; Samsonov, D.; Cope, D.; Liesen, J. L.; Li, K.; Seier, L.; Wolf, T.; Berdica, U.; Mohl, V.; Goldie, A. D.; Courville, A.; Sevegnani, K.; Whiteson, S.; and Foerster, J. N. 2026. Evolution Strategies at the Hyperscale. In *Proceedings of ICML 2026*, volume 306.

Shao, Z.; Wang, P.; Zhu, Q.; Xu, R.; Song, J.; Bi, X.; Zhang, H.; Zhang, M.; Li, Y. K.; Wu, Y.; and Guo, D. 2024. DeepSeekMath: Pushing the Limits of Mathematical Reasoning in Open Language Models. *CoRR*, abs/2402.03300.

Sheng, G.; Zhang, C.; Ye, Z.; Wu, X.; Zhang, W.; Zhang, R.; Peng, Y.; Lin, H.; and Wu, C. 2025. HybridFlow: A Flexible and Efficient RLHF Framework. In *Proceedings of EuroSys 2025*, 1279–1297. ACM.

Sun, Z.; Dang, S.; Dai, G.; and Ye, H. 2026. ESSAM: A Novel Competitive Evolution Strategies Approach to Reinforcement Learning for Memory Efficient LLMs Fine-Tuning. *CoRR*, abs/2602.01003.

Trivedi, H.; Balasubramanian, N.; Khot, T.; and Sabharwal, A. 2022. MuSiQue: Multihop Questions via Single-hop Question Composition. *Trans. Assoc. Comput. Linguist.*, 10: 539–554.

Wierstra, D.; Schaul, T.; Glasmachers, T.; Sun, Y.; Peters, J.; and Schmidhuber, J. 2014. Natural Evolution Strategies. *J. Mach. Learn. Res.*, 15(27): 949–980.

Wu, J.; Lu, N.; Liu, S.; Wang, K.; Yang, Y.; Qing, L.; and Tang, K. 2026. Train at Moving Edge: Online-Verified Prompt Selection for Efficient RL Training of Large Reasoning Model. *CoRR*, abs/2603.25184.

Yang, P.; Zhang, H.; Yu, Y.; Li, M.; and Tang, K. 2022. Evolutionary reinforcement learning via cooperative coevolutionary negatively correlated search. *Swarm Evol. Comput.*, 68: 100974.

Yang, Z.; Qi, P.; Zhang, S.; Bengio, Y.; Cohen, W. W.; Salakhutdinov, R.; and Manning, C. D. 2018. HotpotQA: A Dataset for Diverse, Explainable Multi-hop Question Answering. In *Proceedings of EMNLP 2018*, 2369–2380.

Yang, Z.; Tang, K.; and Yao, X. 2008. Large Scale Evolutionary Optimization Using Cooperative Coevolution. *Inf. Sci.*, 178(15): 2985–2999.

Yao, S.; Zhao, J.; Yu, D.; Du, N.; Shafran, I.; Narasimhan, K. R.; and Cao, Y. 2023. ReAct: Synergizing Reasoning and Acting in Language Models. In *Proceedings of ICLR 2023*.

Zhang, G.; Geng, H.; Yu, X.; Yin, Z.; Zhang, Z.; Tan, Z.; Zhou, H.; Li, Z.; Xue, X.; Li, Y.; Zhou, Y.; Chen, Y.; Zhang, C.; Fan, Y.; Wang, Z.; Huang, S.; Velez, F. P.; Liao, Y.; Wang, H.; Yang, M.; Ji, H.; Wang, J.; Yan, S.; Torr, P.; and Bai, L. 2026. The Landscape of Agentic Reinforcement Learning for LLMs: A Survey. *Trans. Mach. Learn. Res.*, 2026.

Supplementary Material for Cooperative Coevolution for Resource-Constrained Agentic LLM Post-Training

Task	Dataset	Source split	Role	Examples used
Math	MATH	Training	Post-training	6,748
	MATH	Training	Validation	750
	AIME 2024	Test	Evaluation	30
	AIME 2025	Test	Evaluation	30
	GSM8K	Test	Evaluation	1,319
	MATH-500	Test	Evaluation	500
	MATH-Test	Test	Evaluation	5,000
QA	HotpotQA	Training	Post-training	89,697
	HotpotQA	Training	Validation	750
	HotpotQA	Validation	Evaluation	5,000
	2Wiki	Development	Evaluation	5,000
	MuSiQue	Development	Evaluation	2,417

Table S5: Datasets, source splits, and numbers of examples used for post-training, validation, and evaluation.

A Additional Experimental Details

A.1 Models and Datasets

Models. We use Qwen3.5-4B (Qwen Team 2026) as the initial model for every post-training experiment. Qwen3.5-4B without post-training serves as the common reference, while Qwen3.5-9B without post-training is included only as a larger-model reference on the math task.

Math Data. We construct the post-training data from the 7,500-example training split of MATH (Hendrycks et al. 2021). A subject-stratified random split holds out 750 examples for validation. After removing two unusable examples from the remaining data, 6,748 examples are used for post-training. The training and validation data strictly exclude MATH-Test. We evaluate on AIME 2024,³ AIME 2025,⁴ GSM8K (Cobbe et al. 2021), MATH-500 (Lightman et al. 2024), and MATH-Test. MATH-500 is a curated 500-example subset of the 5,000-example MATH-Test set rather than an independent test set.

QA Data. For the QA task, we hold out 750 examples from the HotpotQA training split (Yang et al. 2018) for validation and remove them from the post-training data, leaving 89,697 training examples. Evaluation uses 5,000 randomly selected examples from the HotpotQA validation split, 5,000 randomly selected examples from the 2Wiki development split (Ho et al. 2020), and the complete 2,417-example MuSiQue development split (Trivedi et al. 2022). Table S5 summarizes the data used in both tasks.

A.2 Agent Environment

The two tasks use the same tool-using agent equipped with Local Wiki Search over the Wiki18 corpus (Jin et al. 2025)

³https://huggingface.co/datasets/HuggingFaceH4/aime_2024

⁴https://huggingface.co/datasets/test-time-compute/aime_2025

and a Python Sandbox. The search tool returns evidence from the local corpus, while the sandbox supports commonly used Python packages for computation and answer verification. For each math problem, the agent may invoke Local Wiki Search at most three times and the Python Sandbox at most five times. The corresponding limits for QA are five search calls and five Python calls. The maximum context length is 128K. Invalid tool calls are retried; if a tool or context limit is reached, an available final answer is still extracted and verified, whereas an interaction without a valid answer is treated as unanswered.

A.3 Training Rewards and Answer Verification

Answer Extraction. We inspect assistant outputs in reverse order and first locate the most recent complete answer block, delimited by `<answer>` and `</answer>`. Within this block, we use the last boxed or framed value when present; otherwise, we use the entire block content. When no answer block is present, we extract the last boxed value from the full output. If neither form is found, the complete output is used as the fallback prediction. Correctness verification and format scoring are computed separately, so a fallback prediction can still be verified while incurring a format penalty.

Math Reward. The extracted math answer is normalized with regular-expression-based cleaning and then checked for mathematical equivalence using SymPy (Meurer et al. 2017). Let $a_{\text{math}} \in \{0, 1\}$ denote the resulting correctness indicator. The training reward is

$$R_{\text{math}} = a_{\text{math}} + r_t + r_f. \quad (14)$$

The tool-use bonus is $r_t = 0.1$ when $a_{\text{math}} = 1$ and at least one tool was used, and $r_t = 0$ otherwise.

QA Reward. The QA base score is the token-overlap F1 defined in Section A.4. Its training reward is

$$R_{\text{QA}} = \text{F1} + r_t + r_f, \quad (15)$$

where $r_t = 0.1$ if $\text{F1} > 0$ and at least one tool was used, and $r_t = 0$ otherwise.

Format Reward. Both tasks use the same format reward. We set $r_f = 0$ when the output contains both an `<answer>` block and a boxed answer, $r_f = -0.5$ when either is missing, and $r_f = -1$ when both are missing.

A.4 Evaluation Protocol

Math Evaluation. Validation accuracy is computed on the 750 held-out problems using one generation per problem with temperature 0. Test-time decoding uses temperature 0.3 and top- $p = 1.0$. For GSM8K, MATH-500, and MATH-Test, we generate 32 outputs per problem and report pass@ k for $k = 1, \dots, 16$. For AIME 2024 and AIME 2025, we generate 256 outputs per problem and report $k = 1, \dots, 128$.

Setting	Value
Prompt batch size	64
Full-parameter GRPO learning rate	1×10^{-6}
LoRA-based GRPO learning rate	5×10^{-6}
GRPO group / mini-batch size	8 / 32
GRPO KL coefficient	1×10^{-3}
GRPO PPO ratio clipping range	[0.8, 1.2]
GRPO group-sampling temperature	1.0
LoRA rank / scaling factor	32 / 64
ES/CoPES population N	40
ES/CoPES σ / α	$1 \times 10^{-3} / 5 \times 10^{-4}$
CoPES K / N_k	4 / 10
CoPES σ_k	2×10^{-3}

Table S6: Hyperparameters used in the experiments.

QA Evaluation. QA validation uses one generation per problem with temperature 0, and the validation metric is mean F1 over the 750 held-out problems. Test-time decoding uses temperature 0.3, top- $p = 1.0$, and 32 outputs per problem. We normalize predictions and references by lower-casing, removing English articles, stripping and collapsing whitespace, and applying Unicode case and diacritic normalization; punctuation is retained. Accuracy is one when a normalized ground-truth answer is a contiguous substring of the normalized prediction, exact match (EM) requires equality, and F1 is computed from token overlap. With multiple ground-truth answers, each metric takes its maximum over the references. Each reported Accuracy, EM, or F1 value is first averaged over the 32 outputs for a problem and then averaged across problems.

Pass@ k and Repeated Sampling. For a problem with n generated outputs, of which c are correct, we use the unbiased estimator (Chen et al. 2021)

$$\text{pass@ } k = 1 - \frac{\binom{n-c}{k}}{\binom{n}{k}}. \quad (16)$$

Math correctness is determined by the verifier described above. The additional QA pass@ k results in Section E use the binary Accuracy criterion.

A.5 Training Hyperparameters

Table S6 lists the method-specific settings. Following prior work, we use recommended learning-rate settings for the GRPO variants (Shao et al. 2024; Devulapalli, Satsangi, and Narayan 2025) and established settings for ES-based LLM post-training (Qiu et al. 2026; Liu et al. 2025). We determine $K = 4$ before benchmark test evaluation and use the same method-specific hyperparameters on the math and QA tasks. Standard ES and CoPES use greedy generation with temperature 0 during perturbed-model evaluation. All three post-training methods in the QA experiment run for 48 steps. Each reported training configuration is run once. Dataset splitting and evaluation-set subsampling use seed 1088, while the ES-based and GRPO-based methods use training seeds 33 and 42, respectively.

A.6 Frameworks and Compute Environment

The servers use Intel Xeon Gold 6338 or AMD EPYC 7713 CPUs and run Ubuntu 22.04 with Python 3.11 and CUDA 13. Standard ES and CoPES use vLLM 0.20.2 (Kwon et al. 2023), while both GRPO variants use VeRL 0.8.0 (Sheng et al. 2025).

A.7 Hardware-Feasibility Configuration

For the hardware-feasibility tests reported in the main paper, we enable all applicable memory optimizations supported by each framework. Full-parameter GRPO uses gradient checkpointing together with parameter, optimizer, and activation offloading. LoRA-based GRPO uses gradient checkpointing and parameter and optimizer offloading; activation offloading is unavailable for this configuration. Fused kernels are unavailable for Qwen3.5-4B in the evaluated VeRL configuration. Standard ES and CoPES use the standard vLLM inference path with KV caching, without additional framework-level GPU-memory optimizations. These settings are enabled in the hardware-feasibility tests and are not algorithmic contributions of this work.

A.8 Memory-Efficient ES Implementation

Seed Replay and Chunking. Following OpenAI-ES (Salimans et al. 2017), each perturbation is represented by a random seed and regenerated during the update. CoPES replays each perturbation in parameter chunks, avoiding storage of full perturbation vectors on either GPU or CPU.

CPU Weight Backup. Rather than restoring the unperturbed model by subtracting a replayed perturbation, CoPES backs up the pre-perturbation weights in CPU memory and directly restores them after each perturbed-model evaluation. This design avoids residual numerical errors caused by the non-reversibility of floating-point addition and subtraction and ensures that every perturbation is evaluated from the same pre-update model.

Partition Replay. CoPES also stores the random seed used to construct each parameter partition. During the update, it replays the chunk-to-subspace assignment from this seed instead of retaining a parameter-level assignment mask. Seed replay, chunked processing, CPU weight backup, and partition replay support memory-efficient execution; they do not alter the algorithmic update described in the main paper.

A.9 GPU-Hour Profiling

GPU-hour profiling measures training-step time only. We profile CoPES and full-parameter GRPO for 32 steps on the same $8 \times$ RTX 5880 Ada server as representatives of the ES-based and RL-based method classes, respectively, and extrapolate their average per-step costs to each method’s reported training steps. A profiled step includes perturbed-model or policy evaluation, tool execution, reward computation, and the parameter update; time spent on validation is excluded. This class-level extrapolation reflects that perturbed-model or policy evaluation dominates per-step time, while partition replay and parameter updates contribute negligibly to total GPU-hours in our measurements.

Accounted component	Full-parameter GRPO	LoRA-based GRPO	Standard ES / CoPES
GPU-resident model and training states	69.44 GB	18.55 GB	8.68 GB
Parameter gradients	Included	LoRA parameters only	None
Optimizer states	Included	LoRA parameters only	None
Backpropagation activations	Included	Included	None
Logits and entropy-related tensors	Included	Included	Not retained for backward
Context-dependent term	$3L$ GB	$3L$ GB	$0.032L$ GB
Total	$69.44 + 3L$ GB	$18.55 + 3L$ GB	$8.68 + 0.032L$ GB

Table S7: Components included in the theoretical GPU memory accounting. L denotes context length in K tokens.

B Theoretical GPU Memory Accounting

The memory analysis in the main paper estimates algorithm-level GPU memory requirements for Qwen3.5-4B. It accounts for model weights, parameter gradients, optimizer states, backpropagation activations, logits and entropy-related tensors, and inference KV caches. FlashAttention (Dao et al. 2022) and KV caching are assumed. To isolate the memory implied by each optimization paradigm, the analysis excludes gradient checkpointing, offloading, fused kernels, and other framework-specific memory optimizations. The GRPO estimates cover actor training only and omit a separately resident reference policy and generation engine, making them conservative with respect to the RL-based methods. Standard ES and CoPES have the same theoretical GPU memory requirements.

Fixed States. Qwen3.5-4B contains $P = 4,659,865,088$ parameters. Full-parameter GRPO retains FP32 model weights, parameter gradients, and two FP32 AdamW states, giving

$$\frac{(4 + 4 + 8)P}{2^{30}} = 69.44 \text{ GB.} \quad (17)$$

For LoRA-based GRPO with rank 32, the frozen FP32 base model occupies 17.36 GB. The 80,150,528 LoRA parameters and their gradients and optimizer states add 1.19 GB, for a fixed total of 18.55 GB. ES-based methods retain only the BF16 model weights on GPU:

$$\frac{2P}{2^{30}} = 8.68 \text{ GB.} \quad (18)$$

The pre-perturbation backup used by CoPES resides in CPU memory and therefore does not add another GPU-resident model copy.

Context-Dependent Memory. For both GRPO variants, the dominant context-dependent terms are the full vocabulary logits, entropy/softmax tensors, and activations retained for backpropagation. Their combined lower-bound estimate is approximately 3 GB per K tokens. LoRA reduces trainable parameter states but does not remove the need to backpropagate through the model, so it has essentially the same context-dependent slope. ES and CoPES perform forward-only generation and retain neither parameter gradients nor backpropagation activations. Qwen3.5-4B has eight full-attention layers, four KV heads, and a head dimension of 256, yielding 32 KiB of BF16 KV cache per token, or approximately 0.032 GB per K tokens.

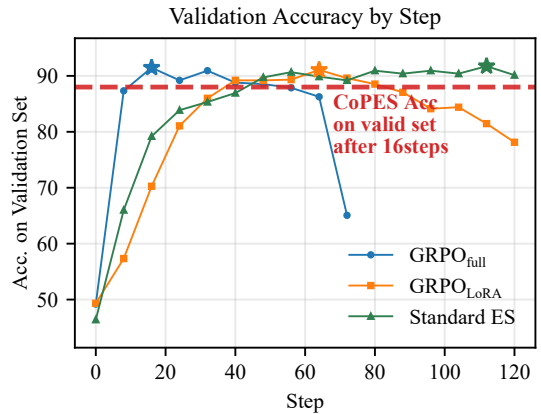


Figure S4: Validation accuracy during math post-training. Stars mark the best observed validation checkpoints for full-parameter GRPO, LoRA-based GRPO, and standard ES. The horizontal dashed line marks CoPES validation accuracy at its 16-step fixed-budget endpoint rather than a CoPES training curve.

Conservative Rounding. The GRPO context-dependent coefficient is rounded near the lower end of its component-wise estimate, whereas the exact ES KV-cache coefficient of 0.03125 GB per K tokens is rounded upward to 0.032. These choices underestimate rather than exaggerate the relative memory advantage of ES. The values are theoretical accounting estimates rather than measured peak GPU memory and intentionally omit runtime-specific overheads.

Final Estimates. The resulting theoretical requirements are

$$M_{\text{full}}(L) = 69.44 + 3L \text{ GB,} \quad (19)$$

$$M_{\text{LoRA}}(L) = 18.55 + 3L \text{ GB,} \quad (20)$$

$$M_{\text{ES/CoPES}}(L) = 8.68 + 0.032L \text{ GB.} \quad (21)$$

At a context length of 128K, these expressions give 453.44 GB for full-parameter GRPO, 402.55 GB for LoRA-based GRPO, and 12.78 GB for standard ES or CoPES. Thus, even the LoRA-based estimate is more than $30\times$ the ES estimate, while full-parameter GRPO requires approximately $35\times$ as much memory.

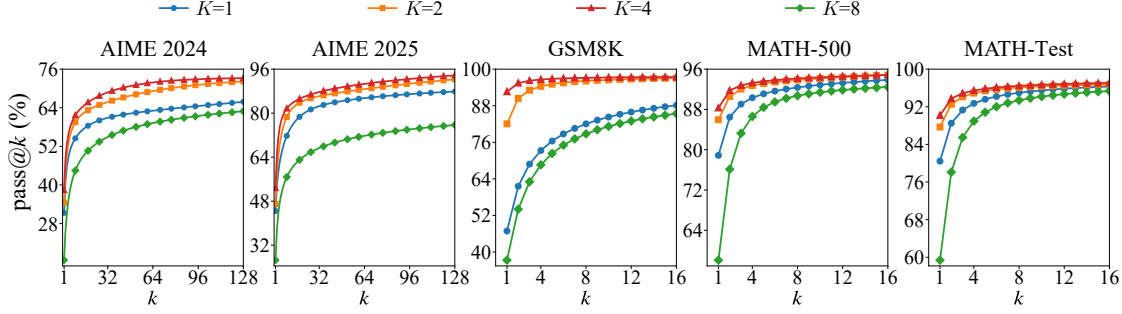


Figure S5: Pass@ k for $K \in \{1, 2, 4, 8\}$, where K is the number of parameter subspaces. The total population remains $N = 40$, with $N_k = N/K$ and $\sigma_k = \sqrt{K}\sigma$.

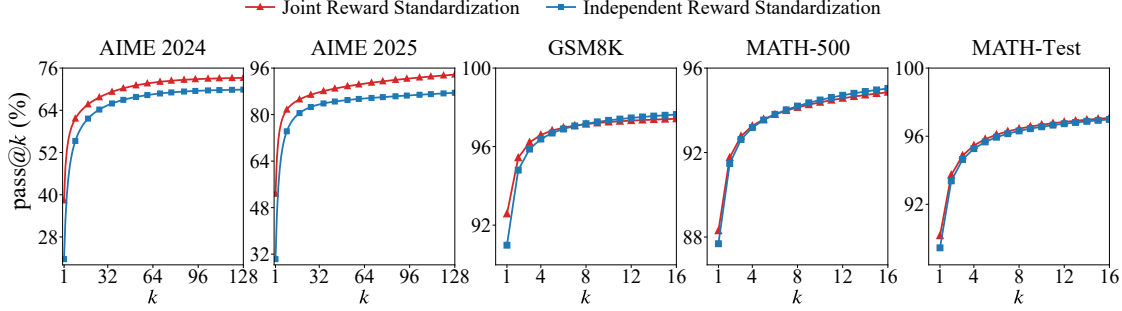


Figure S6: Pass@ k with joint reward standardization and independent reward standardization. Both variants use $K = 4$ and otherwise identical settings.

C Validation Curves and Checkpoint Selection

For methods whose training trajectories extend beyond their highest observed validation performance, we report the corresponding best observed checkpoint. For methods run only to a common fixed budget, we report the budget-end checkpoint without characterizing their subsequent training behavior.

Math Validation. Figure S4 shows the complete validation trajectories used to identify the best observed validation checkpoints reported in the main paper: step 16 for full-parameter GRPO, step 64 for LoRA-based GRPO, and step 112 for standard ES. CoPES is evaluated only within the common 16-step budget, so its horizontal dashed line represents the validation accuracy reached at that endpoint. Standard ES first exceeds this reference level at step 48, requiring three times as many training steps as the CoPES fixed-budget run.

QA Validation. Figure S8 reports mean validation F1. Full-parameter GRPO reaches its highest observed validation F1 at step 48, which determines the common 48-step QA budget used in the main comparison. The CoPES dashed line marks its validation F1 at this fixed-budget endpoint; its behavior beyond 48 steps was not evaluated. Standard ES reaches the CoPES reference level at step 64; its later checkpoints are shown only to characterize the validation trajectory and are not used in the 48-step main comparison.

Post-Peak Behavior. The two GRPO curves on math and the full-parameter GRPO curve on QA decline after reaching their highest observed validation performance. We observed repetitive reasoning loops in some later GRPO outputs, which may contribute to this behavior, but this explanation remains a hypothesis rather than an established causal result.

D Complete Ablation Results

All ablations use a 16-step fixed budget and a total population of $N = 40$. In the subspace-count ablation, $N_k = N/K$ and $\sigma_k = \sqrt{K}\sigma$; the other ablations retain the main CoPES settings except for the factor being evaluated.

Number of Subspaces. Figure S5 extends the pass@1 ablation in the main paper to the complete pass@ k curves. Performance generally improves as K increases from 1 to 4, while $K = 8$ performs substantially worse. At $K = 8$, each subspace receives only $N_k = 5$ perturbations, and the model is divided into more parameter groups. The smaller population per subspace and weaker preservation of parameter interactions may both reduce update quality.

Joint Reward Standardization. Figure S6 compares the proposed joint standardization with independently standardizing the rewards within each subspace. Joint standardization gives higher pass@1 on all five benchmarks. On GSM8K and MATH-500, the curves approach saturation and cross slightly

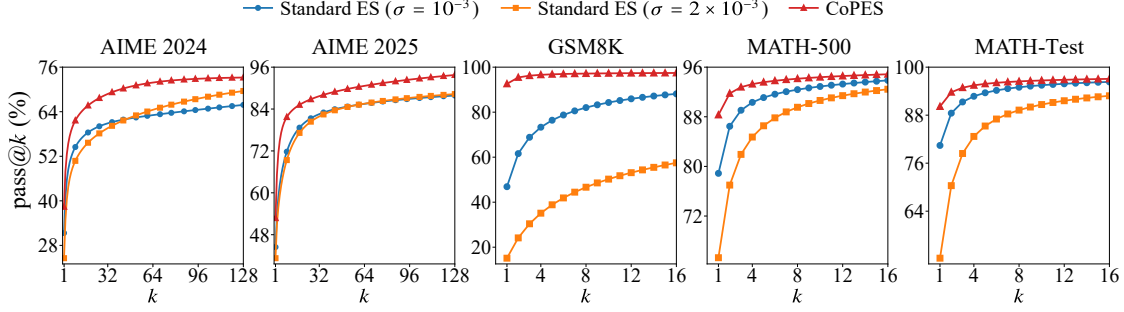


Figure S7: Pass@ k for standard ES with $\sigma = 10^{-3}$, standard ES with $\sigma = 2 \times 10^{-3}$, and CoPES. The second setting matches the per-coordinate perturbation scale used by CoPES.

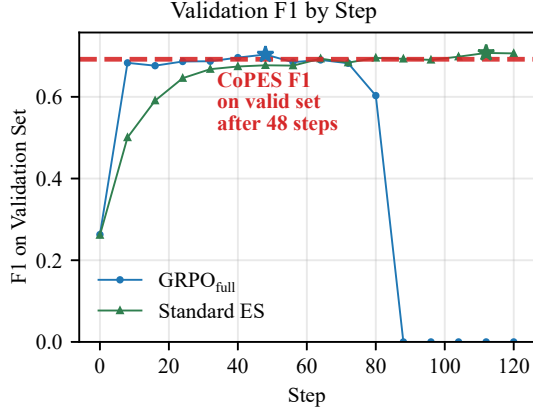


Figure S8: Validation F1 during QA post-training. The horizontal dashed line is CoPES validation F1 at its 48-step fixed-budget endpoint rather than a CoPES training curve.

at larger k ; these small pointwise differences do not establish a consistent high- k advantage for either normalization mode. Overall, the results support joint standardization at the primary pass@1 operating point without claiming uniform dominance at every k .

Perturbation-Scale Control. CoPES uses $\sigma_k = 2\sigma$ when $K = 4$. Figure S7 therefore compares CoPES with standard ES at both σ and 2σ under the same fixed budget. Increasing the standard-ES perturbation scale degrades its performance across the benchmarks rather than reproducing the CoPES improvement. The benefit of CoPES therefore cannot be attributed solely to its larger per-coordinate perturbation scale.

E Additional Results on the QA Task

The main paper reports Accuracy, EM, and F1 averaged over the 32 evaluation outputs per problem. Here, we additionally characterize repeated-sampling behavior with Accuracy-based pass@ k . A generated output is counted as correct when its binary Accuracy equals one, and Equation 16 is applied to the same 32 outputs used to compute the main QA table.

Figure S9 reports $k = 1, \dots, 16$. By construction, pass@1 exactly matches the Accuracy values in the main QA table. CoPES exceeds standard ES across the three curves, extend-

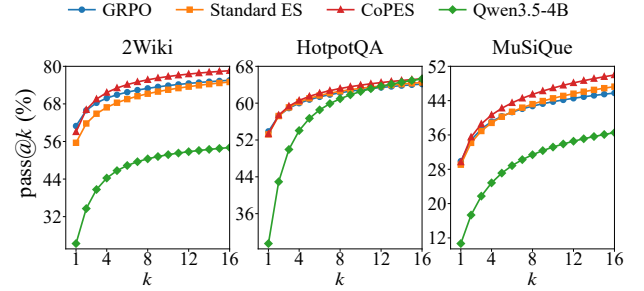


Figure S9: Accuracy-based Pass@ k on 2Wiki, HotpotQA, and MuSiQue. Qwen3.5-4B is the reference model without post-training; the other methods use the common 48-step QA budget.

ing its advantage to repeated sampling on the QA task. Compared with full-parameter GRPO, CoPES starts slightly lower at pass@1 but achieves higher pass@ k for $k \geq 2$ on 2Wiki and MuSiQue. The two methods remain close on HotpotQA, where CoPES also becomes slightly higher as k increases. These curves supplement rather than replace the Accuracy, EM, and F1 comparisons in the main paper.